Document number: P0441R0
Date: 2016-10-17
Project: C++ Extensions for Ranges, Library Evolution Working Group and Library Working Group
Reply-to: Casey Carter <Casey@Carter.net>
Eric Niebler <Eric.Niebler@gmail.com>

Ranges: Merging Writable and MoveWritable

1 Introduction

This paper presents a design change to N4560, the working paper for the Technical Specification for C++ Extensions for Ranges (the "Ranges TS")[3]. This work has already been speculatively integrated - along with the proposals in P0370 "Ranges TS Design Updates Omnibus"[1] - with the text of the working paper. The resulting document is P0459 "C++ Extensions for Ranges: Speculative Combined Proposals"[4]. We briefly discuss how the design of the `Writable` and `MoveWritable` concepts impacts the specification of algorithms, and propose to correct the problem by merging the two into a single concept.

1.1 Motivation

The current Ranges TS Working Paper (N4560) includes two concepts used to describe relationships between a readable and a writable iterator, `MoveWritable` and `Writable`:

```
template <class Out, class T>
concept bool MoveWritable() {
  return Semiregular<Out>() &&
    requires(Out o, T t) {
      *o = std::move(t);
    }
}

template <class Out, class T>
concept bool Writable() {
  return MoveWritable<Out, T>() &&
    requires(Out o, const T t) {
      *o = t;
    }
}
```

with similar semantics for the assignment expressions, except that `t` is left unchanged by the assignment in `Writable` whereas `MoveWritable` leaves `t` in a valid but unspecified state. These concepts describe how an algorithm transfers values into its outputs. We could implement, e.g., `fill_n` as:

```
template <class T, WeaklyIncrementable Out>
  requires Writable<Out, T>()
void fill_n(Out out, difference_type_t<Out> n, const T& value) {
  for (; n > 0; --n, ++out) {
    *out = value;
  }
}
```

and know that the expression `*out = value;` has the effect of "copying" `value` into the output sequence thanks to the `Writable` constraint.

The existence of the two distinct concepts results in a partitioning of algorithms into two sets: those that are based on `MoveWritable` or refinements thereof that always move, and those based on `Writable` or refinements thereof that could copy or move or do *both*. In practice, algorithms in the `Writable`/`IndirectlyCopyable` are always specified to copy; passing `move_iterator`s to these algorithms results in either (a) a diagnosis that the program is ill-formed when the value type of the iterator is not copyable, or (b) undefined behavior when the value type of the iterator *is* copyable, but the expression `*out = *in` does not leave the value of `*in` unchanged as required by the semantic constraints of `Writable`. As a result, we need distinct "copy" and "move" variants of algorithms resulting in proliferation. In N4560, for example, there are `partition_move` and `merge_move` algorithms as a result of this issue.

The need to choose between `Writable` and `MoveWritable` also complicates generic code. Consider a simplified variant of the `transform` algorithm:

```
template<InputIterator I, Sentinel<I> S, WeaklyIncrementable O, class F>
  requires Writable<O, indirect_result_of_t<F&(I)>>()
void transform(I first, S last, O result, F op);
```

Presumably, implementation of this algorithm requires an assignment expression like `*result = op(*first);`. We're again constrained (pun intended) by the choice of concepts: if `op(*first)` returns an rvalue of type `T`, `Writable` unnecesarily requires `T` to be copyable. If `op(*first)` returns an lvalue, `MoveWritable` would allow it to be modified by the assignment. None of the concepts defined in N4560 enable us to constrain `transform` in a manner that is compatible with its semantics in C++14.

Separately, there is a problem using the existing concepts to constrain algorithms that create temporaries. Such algorithms typically require the ability to:

1. construct a temporary object of an iterator's value type from a dereferenced iterator
2. replace the stored value of a temporary by assigning from a dereferenced iterator
3. transfer values between multiple temporaries
4. transfer a value from a temporary into an output sequence

These algorithms are underconstrained in N4560. Wrapping the requirements up into convenient named packages would ease the specification of algorithms.

2 Proposed Design

We propose coalescing the existing `Writable` and `MoveWritable` concepts into a single `Writable` concept, and adding concepts `IndirectlyMovableStorable` and `IndirectlyCopyableStorable` for constraining algorithms that use temporaries.

2.1 Coalescing `Writable` and `MoveWritable`

```
template <class O, class T>
concept bool Writable() {
  return Semiregular<O>() && requires(O o, T&& t) {
    *o = std::forward<T>(t); // not required to be equality-preserving
  };
}
```

Intuitively, `Writable<O, T>()` is satisfied when an expression with type and value category `T` can be assigned through an iterator with type `O`. For a possibly-cv-qualified non-reference type `T`, `Writable<O, T>` is roughly equivalent to N4560's `MoveWritable<O, T>` and `Writable<O, const T>` is roughly equivalent to N4560's `Writable<O, T>`.

Note that since `OutputIterator<I, T>()` requires `Writable<I, T>()` and `OutputRange<R, T>()` requires `OutputIterator<iterator_t<R>, T>()`, the meaning of those two concepts is changed as well. `OutputIterator<I, T>` no longer means "a value of type `T` can be assigned through an iterator of type `I`," but instead "an *expression* of type and value category `T` can be assigned through an iterator of type `I`." We also need to flow the `Writable` change into `IndirectlyCopyable` and `IndirectlyMovable`. We'll define a new type alias `rvalue_reference_t<In>` for the type of `std::move(*i)` to make the symmetry nicely apparent:

```
template <class In>
using rvalue_reference_t = decltype(std::move(declval<reference_t<In&>>()));

template <class In, class Out>
concept bool IndirectlyMovable() {
  return Readable<In>() && Writable<Out, rvalue_reference_t<In>>();
}
```

```
template <class In, class Out>
concept bool IndirectlyCopyable() {
  return Readable<In>() && Writable<Out, reference_t<In>>();
}
```

Note that `IndirectlyCopyable` does not refine `IndirectlyMovable` as in N4560: there are no algorithms that use both copy syntax (`*out = *in;`) and move syntax (`*out = std::move(*in);`) to transfer values between their input and output sequences. Even if the `IndirectlyMovable` requirements hold for every "sane" model of `IndirectlyCopyable`, it seems pointless to invest the compile time to validate the "extra" requirements when the algorithm will never use the additional capabilities they provide.

2.2 `Storable` concept variants

We define `Storable` refinements of the `IndirectlyMovable` and `IndirectlyCopyable` concepts to constrain algorithms that sometimes interject temporaries into the communication of values between their input and output sequences:

```
template <class In, class Out>
concept bool IndirectlyMovableStorable() {
  return IndirectlyMovable<In, Out>() &&
    Writable<Out, value_type_t<In>>() &&
    Movable<value_type_t<In>>() &&
    Constructible<value_type_t<In>, rvalue_reference_t<In>>() &&
    Assignable<value_type_t<In>&, rvalue_reference_t<In>>();
}

template <class In, class Out>
concept bool IndirectlyCopyableStorable() {
  return IndirectlyCopyable<In, Out>() &&
    Writable<Out, const value_type_t<In>&>() &&
    Copyable<value_type_t<In>>() &&
    Constructible<value_type_t<In>, reference_t<In>>() &&
    Assignable<value_type_t<In>&, reference_t<In>>();
}
```

These definitions include the requirements described earlier for constructing and assigning temporaries from the input sequence, moving/copying between temporaries, and writing temporaries into the output sequence. The names of the concepts themselves are a bit of a mouthful, which issue is ameliorated by the fact that they rarely need to be used directly in the algorithm specifications. If we add an `IndirectlyMovableStorable` requirement to the `Permutable` concept, which is used to constrain the many algorithms that mutate their input sequence by swapping values between elements, the only remaining algorithm needing a constraint is `unique_copy`.

2.3 Eliminating `merge_move` and `partition_move`

With the changes to `Writable` in place, there is no longer a need for separate "copy" and "move" algorithm variants. It is again possible obtain the effect of a "move" algorithm by wrapping an iterator pair in `move_iterator`s and calling the "copy" algorithm. We therefore propose to remove the `merge_move` and `partition_move` algorithms that were added to the TS to workaround this issue.

2.3.1 Addition of `move_sentinel`

But wait - what if I have an [iterator, sentinel) range instead of an iterator pair? One cannot wrap a sentinel in a `move_iterator`. There are at least three potential solutions:

1. Define a `move_sentinel<S>` wrapper class that satisfies `Sentinel<move_sentinel<S>, move_iterator<I>>()` when `Sentinel<S, I>()` is satisfied. For ease of use, and consistency of interface, define a `make_move_sentinel` deducing helper function. This approach maintains a clear distinction between the adapted range and the underlying range. The definition of `foo` becomes:

   ```
   template <InputIterator I, Sentinel<I> S>
   void foo(I first, S last) {
     bar(make_move_iterator(first), make_move_sentinel(last));
   }
   ```

   This approach is straightforward, and while perhaps lacking elegance, gets the job done.

2. Define operator overloads for `==` and `!=` that accept `move_iterator<I>` and `Sentinel<I>` so that move iterators can use the sentinels of their base iterators directly:

```
Sentinel{S, I}
bool operator==(const move_iterator<I>& i, const S& s) {
  return i.current_ == s;
}
// Define != similarly, and the symmetric overloads.
```

`foo` can be simpler since it passes the sentinel directly:

```
template <InputIterator I, Sentinel<I> S>
void foo(I first, S last) {
  bar(make_move_iterator(first), last);
}
```

This approach requires less syntax at the callsite. It is a bit lax in that it allows comparing adapted iterators with unadapted sentinels, potentially creating confusion in code readers about which iterators/sentinels are adapted and which are from the base range.

3. Ignore the problem until Ranges TS2 comes along with support for view adaptors, and the solution will look something like:

```
template <InputIterator I, Sentinel<I> S>
void foo(I first, S last) {
  bar(make_iterator_range(first, last) | view::move);
}
// Or even better:
void foo(InputRange&& rng) {
  bar(rng | view::move);
}
```

Adapting iterator/sentinel pairs together instead of individually is preferable: it allows the library to check for errors, and to optimize the special case when `last` is also an iterator by producing a bounded range (a range whose `begin` and `end` have the same type).

The view adapter is clearly the preferable solution in the long run, but would require the introduction of too much additional machinery into the current TS to be a viable solution in the short-term. Implementation experience using the underlying sentinels directly with the move iterators was not positive: it results in less clear code, and is fragile in the face of poorly constrained iterators/sentinels. We therefore propose the first approach as an interim solution to this problem.

3 Technical Specifications

Rename the section [iterators.movewritable] “Concept MoveWritable” to [iterators.writable] “Concept Writable” and replace its content with:

The `Writable` concept describes the requirements for writing a value into an iterator’s referenced object.

```
  template <class Out, class T>
  concept bool Writable() {
    return Semiregular<Out>() &&
      requires(Out o, T&& t) {
        *o = std::forward<T>(t); // not required to be equality preserving
      };
  }
```

Let `E` be an expression such that `decltype(E)` is `T`, and let `o` be a dereferenceable object of type `Out`. Then `Writable<Out, T>()` is satisfied if and only if

- If `Readable<Out>() && Same<value_type_t<Out>, decay_t<T>>()` is satisfied, then `*o` after the assignment is equal to the value of `E` before the assignment.

After evaluating the assignment expression, `o` is not required to be dereferenceable.

If `E` is an xvalue, the resulting state of the object it denotes is unspecified. [Note: The object must still meet the requirements of any library component that is using it. The operations listed in those requirements must work as specified whether the object has been moved from or not. —end note]

Remove the following sections [iterators.writable], [iterators.indirectlymovable], and [iterators.indirectlycopyable].

Relocate [iterators.indirectlyswappable] to before [commonalgoreq.indirectlycomparable]. Replace its stable name (and all references thereto) with [commonalgoreq.indirectlyswappable].

Replace the content of [commonalgoreq.general] with:

> There are several additional iterator concepts that are commonly applied to families of algorithms. These group together iterator requirements of algorithm families. There are three relational concepts that specify how element values are transferred between `Readable` and `Writable` types: `IndirectlyMovable`, `IndirectlyCopyable`, and `IndirectlySwappable`. There are three relational concepts for rearrangements: `Permutable`, `Mergeable`, and `Sortable`. There is one relational concept for comparing values from different sequences: `IndirectlyComparable`.
>
> [Note: The `equal_to<>` and `less<>` function types used in the concepts below impose additional constraints on their arguments beyond those that appear explicitly in the concepts' bodies. `equal_to<>` requires its arguments satisfy `EqualityComparable`, and `less<>` requires its arguments satisfy `StrictTotallyOrdered`. —end note]

Immediately thereafter, add a new section [commonalgoreq.indirectlymovable] "Concept IndirectlyMovable", replacing all references to [iterators.indirectlymovable] with [commonalgoreq.indirectlymovable] - and content:

> The `IndirectlyMovable` concept specifies the relationship between a `Readable` type and a `Writable` type between which values may be moved.

```
template <class In>
using rvalue_reference_t =
  decltype(std::move(declval<reference_t<In>>()));

template <class In, class Out>
concept bool IndirectlyMovable() {
  return Readable<In>() &&
    Writable<Out, rvalue_reference_t<In>>();
}
```

> The `IndirectlyMovableStorable` concept augments `IndirectlyMovable` with additional requirements enabling the transfer to be performed through an intermediate object of the `Readable` type's value type.

```
template <class In, class Out>
concept bool IndirectlyMovableStorable() {
  return IndirectlyMovable<In, Out>() &&
    Writable<Out, value_type_t<In>>() &&
    Movable<value_type_t<In>>() &&
    Constructible<value_type_t<In>, rvalue_reference_t<In>>() &&
    Assignable<value_type_t<In>&, rvalue_reference_t<In>>();
}
```

Immediately thereafter, add a new section [commonalgoreq.indirectlycopyable] "Concept IndirectlyCopyable", replacing all references to [iterators.indirectlycopyable] with [commonalgoreq.indirectlycopyable] - and content:

> The `IndirectlyCopyable` concept specifies the relationship between a `Readable` type and a `Writable` type between which values may be copied.

```
template <class In, class Out>
concept bool IndirectlyCopyable() {
  return Readable<In>() &&
    Writable<Out, reference_t<In>>();
}
```

> The `IndirectlyCopyableStorable` concept augments `IndirectlyCopyable` with additional requirements enabling the transfer to be performed through an intermediate object of the `Readable` type's value type. It also requires the capability to make copies of values.

```
template <class In, class Out>
concept bool IndirectlyCopyableStorable() {
  return IndirectlyCopyable<In, Out>() &&
    Writable<Out, const value_type_t<In>&>() &&
    Copyable<value_type_t<In>>() &&
    Constructible<value_type_t<In>, reference_t<In>>() &&
    Assignable<value_type_t<In>&, reference_t<In>>();
}
```

In [commonalgoreq.permutable], replace the concept definition with:

```
template <class I>
concept bool Permutable() {
  return ForwardIterator<I>() &&
    IndirectlyMovableStorable<I, I>() &&
    IndirectlySwappable<I, I>();
}
```

Remove the section [commonalgoreq.mergemovable].

Add the following declarations to the synopsis of the `<experimental/ranges/iterator>` header in [iterator.synopsis], between the `move_iterator` and `common_iterator` declarations:

```
  template <Semiregular S> class move_sentinel;

  template <class I, Sentinel<I> S>
    bool operator==(
      const move_iterator<I>& i, const move_sentinel<S>& s);
  template <class I, Sentinel<I> S>
    bool operator==(
      const move_sentinel<S>& s, const move_iterator<I>& i);
  template <class I, Sentinel<I> S>
    bool operator!=(
      const move_iterator<I>& i, const move_sentinel<S>& s);
  template <class I, Sentinel<I> S>
    bool operator!=(
      const move_sentinel<S>& s, const move_iterator<I>& i);

  template <class I, SizedSentinel<I> S>
    difference_type_t<I> operator-(
      const move_sentinel<S>& s, const move_iterator<I>& i);
  template <class I, SizedSentinel<I> S>
    difference_type_t<I> operator-(
      const move_iterator<I>& i, const move_sentinel<S>& s);

  template <Semiregular S>
    move_sentinel<S> make_move_sentinel(S s);
```

In [iterators.move]/1 unstrike the text “Some generic algorithms can be called with move iterators to replace copying with moving.” and remove the now-inaccurate editorial note that follows. After the example, add new paragraphs:

> Class template `move_sentinel` is a sentinel adaptor useful for denoting ranges together with `move_iterator`. When an input iterator type `I` and sentinel type `S` satisfy `Sentinel<S, I>()`, `Sentinel<move_sentinel<S>, move_iterator<I>>()` is satisfied as well.
>
> [Example: A `move_if` algorithm is easily implemented with `copy_if` using `move_iterator` and `move_sentinel`:
>
> ```
> template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O,
>           IndirectCallablePredicate<I> Pred>
>   requires IndirectlyMovable<I, O>()
> void move_if(I first, S last, O out, Pred pred)
> {
>   copy_if(move_iterator<I>{first}, move_sentinel<S>{last}, out, pred);
> }
> ```
>
> —end example]

Insert a new subsection [move.sentinel] after [move.iter.nonmember]:

> 24.7.3.4 Class template `move_sentinel` [move.sentinel]

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
  template <Semiregular S>
  class move_sentinel {
  public:
    constexpr move_sentinel();
    explicit move_sentinel(S s);
    move_sentinel(const move_sentinel<ConvertibleTo<S>>& s);
    move_sentinel& operator=(const move_sentinel<ConvertibleTo<S>>& s);
```

```
    S base() const;

  private:
    S last; // exposition only
  };

  template <class I, Sentinel<I> S>
    bool operator==(
      const move_iterator<I>& i, const move_sentinel<S>& s);
  template <class I, Sentinel<I> S>
    bool operator==(
      const move_sentinel<S>& s, const move_iterator<I>& i);
  template <class I, Sentinel<I> S>
    bool operator!=(
      const move_iterator<I>& i, const move_sentinel<S>& s);
  template <class I, Sentinel<I> S>
    bool operator!=(
      const move_sentinel<S>& s, const move_iterator<I>& i);

  template <class I, SizedSentinel<I> S>
    difference_type_t<I> operator-(
      const move_sentinel<S>& s, const move_iterator<I>& i);
  template <class I, SizedSentinel<I> S>
    difference_type_t<I> operator-(
      const move_iterator<I>& i, const move_sentinel<S>& s);

  template <Semiregular S>
    move_sentinel<S> make_move_sentinel(S s);
}}}}
```

24.7.3.5 `move_sentinel` operations [move.sent.ops]

24.7.3.5.1 `move_sentinel` constructors [move.sent.op.const]

```
constexpr move_sentinel();
```

1 Effects: Constructs a `move_sentinel`, value-initializing `last`. If `S` is a literal type, this constructor shall be a `constexpr` constructor.

```
explicit move_sentinel(S s);
```

2 Effects: Constructs a `move_sentinel`, initializing `last` with `s`.

```
move_sentinel(const move_sentinel<ConvertibleTo<S>>& s);
```

3 Effects: Constructs a `move_sentinel`, initializing `last` with `s.last`.

24.7.3.5.2 `move_sentinel::operator=` [move.sent.op=]

```
move_sentinel& operator=(const move_sentinel<ConvertibleTo<S>>& s);
```

1 Effects: Assigns `s.last` to `last`.

24.7.3.5.3 `move_sentinel` comparisons [move.sent.op.comp]

```
template <class I, Sentinel<I> S>
  bool operator==(
    const move_iterator<I>& i, const move_sentinel<S>& s);```
template <class I, Sentinel<I> S>
  bool operator==(
    const move_sentinel<S>& s, const move_iterator<I>& i);
```

1 Effects: Equivalent to `i.current == s.last`.

```
template <class I, Sentinel<I> S>
  bool operator!=(
    const move_iterator<I>& i, const move_sentinel<S>& s);
template <class I, Sentinel<I> S>
  bool operator!=(
    const move_sentinel<S>& s, const move_iterator<I>& i);
```

2 Effects: Equivalent to `!(i == s)`.

24.7.3.5.4 `move_sentinel` non-member functions [move.sent.nonmember]

```
template <class I, SizedSentinel<I> S>
  difference_type_t<I> operator-(
    const move_sentinel<S>& s, const move_iterator<I>& i);
```

1 Effects: Equivalent to `s.last - i.current`.

```
template <class I, SizedSentinel<I> S>
  difference_type_t<I> operator-(
    const move_iterator<I>& i, const move_sentinel<S>& s);
```

2 Effects: Equivalent to `i.current - s.last`.

```
template <Semiregular S>
move_sentinel<S> make_move_sentinel(S s);
```

3 Returns: `move_sentinel<S>(s)`.

Throughout Clause 25 [algorithms] replace occurrences of `Writable<I, T>`, `OutputIterator<T>`, and `OutputRange<T>` with `Writable<I, const T&>`, `OutputIterator<const T&>`, and `OutputRange<const T&>` respectively.

In the synopsis of the `<experimental/ranges/algorithm>` header in [algorithms.general], in the declarations of `unique_copy`, replace the constraint clause `Copyable<value_type_t<I>>()` (resp. `Copyable<value_type_t<iterator_t<Rng>>>()`) with `IndirectlyCopyableStorable<I, O>()` (resp. `IndirectlyCopyableStorable<iterator_t<Rng>, O>()`). Remove both declarations of `partition_move` and `merge_move`.

In [alg.unique], in the declarations of `unique_copy`, again replace the constraint clauses `Copyable<value_type_t<I>>()` (resp. `Copyable<value_type_t<iterator_t<Rng>>>()`) with `IndirectlyCopyableStorable<I, O>()` (resp. `IndirectlyCopyableStorable<iterator_t<Rng>, O>()`).

In [alg.partitions], strike the declarations of `partition_move` and remove paragraphs 16-19 that specify its behavior.

In [alg.merge], strike the declarations of `merge_move` adn remove paragraphs 6-10 that specify its behavior.

3.1 Implementation Experience

The proposed design changes are implemented in both CMCSTL2, a full implementation of the Ranges TS with proxy extensions[2], and range-v3[5].

References

[1] Carter, C. and Niebler, E. 2016. P0370R1: Ranges TS Design Updates Omnibus.

[2] CMCSTL2: *https://github.com/CaseyCarter/cmcstl2*. Accessed: 2016-05-26.

[3] Niebler, E. and Carter, C. 2015. N4560: Working Draft: C++ Extensions for Ranges.

[4] Niebler, E. and Carter, C. 2016. P0459: C++ Extensions for Ranges: Speculative Combined Proposals.

[5] Range v3: *http://github.com/ericniebler/range-v3*. Accessed: 2014-10-08.